

PRM2

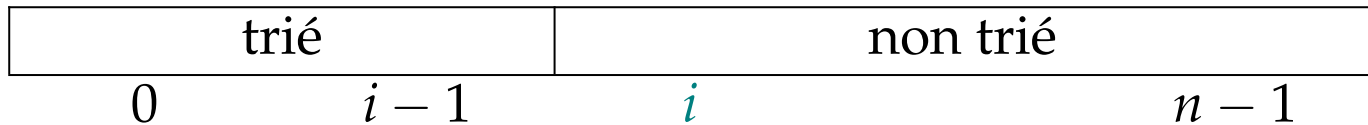
Théorie : Leçon 12bis

Java :

Quelques algorithmes de tri.

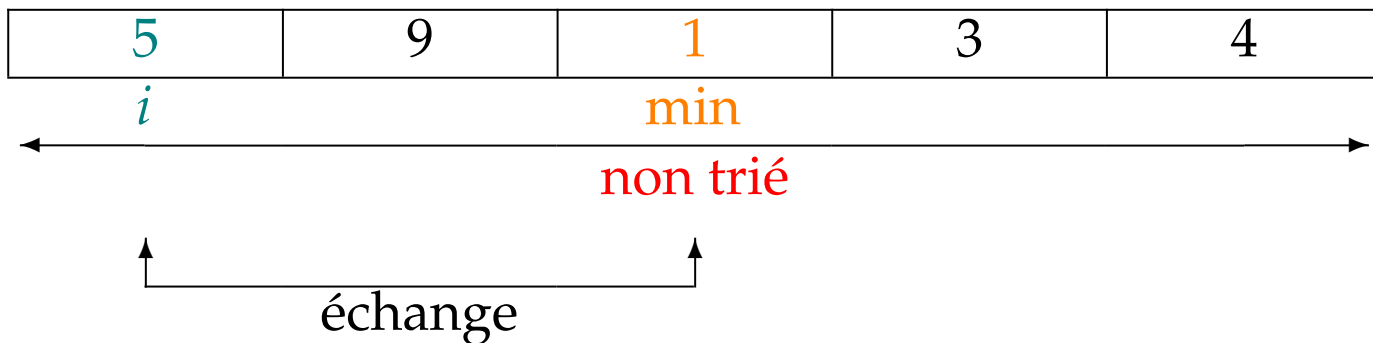
Ranger les éléments d'un tableau dans un certain ordre.

Tri par sélection (I)



A chaque étape i (de 0 à $n - 2$) :

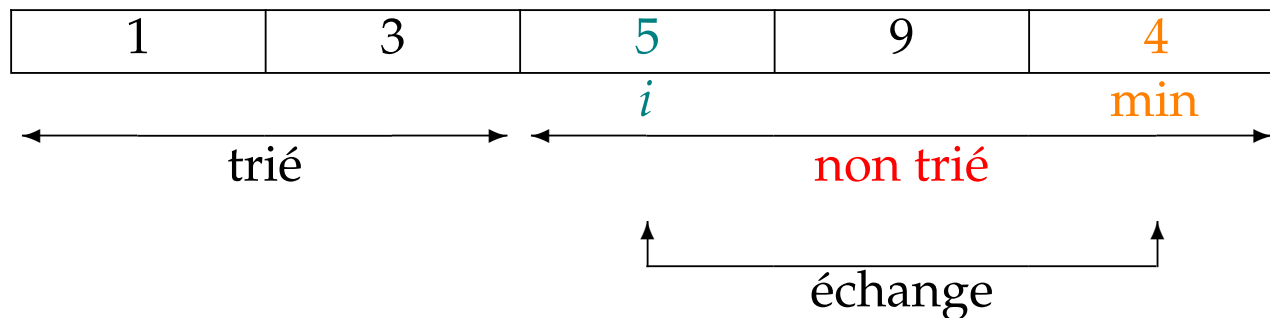
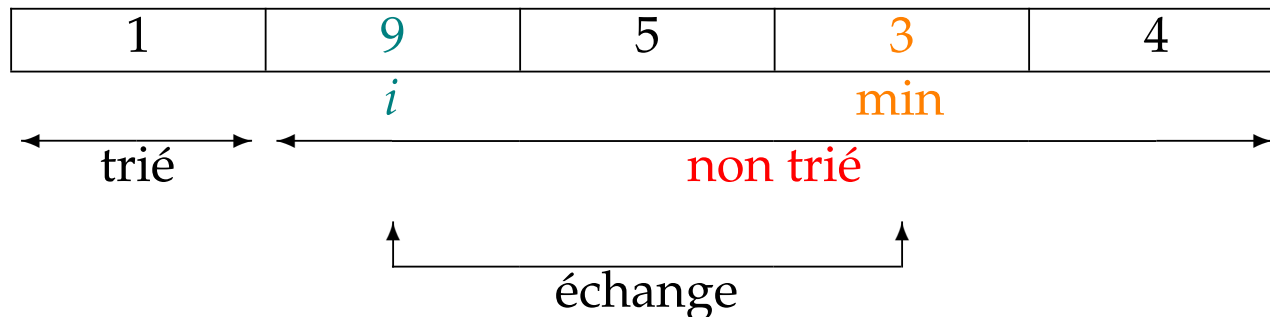
- ① chercher le minimum parmi les éléments non triés
- ② le placer à sa position définitive via un échange



Tri par sélection (2)

A chaque étape i (de 0 à $n - 2$) :

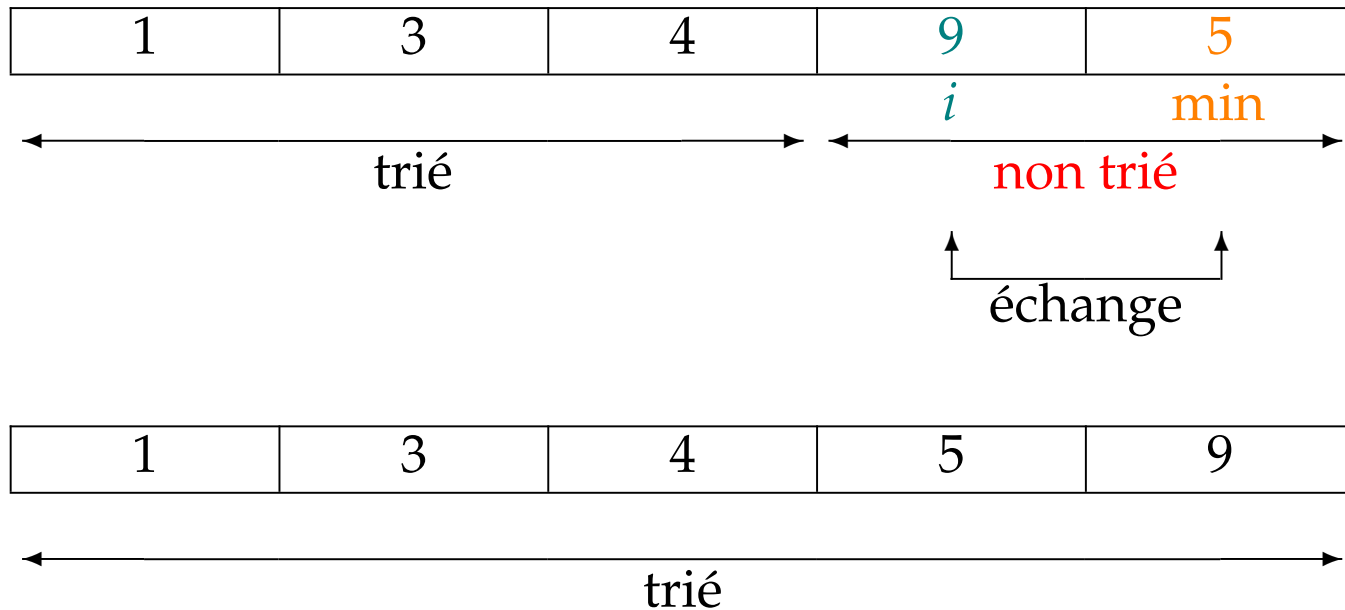
- ① chercher le minimum parmi les éléments non triés
- ② le placer à sa position définitive via un échange



Tri par sélection (3)

A chaque étape i (de 0 à $n - 2$) :

- ① chercher le minimum parmi les éléments non triés
- ② le placer à sa position définitive via un échange

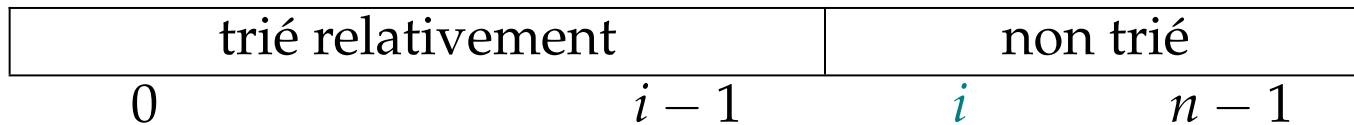


Tri par sélection en Java pour un Array<Integer>

```
public static void triSelection(Array<Integer> tab) {
    //pour chaque élément
    for(int i = 0; i < tab.size(); ++i) {
        //recherche du minimum "à droite"
        int indMin = i;
        for(int j = i + 1; j < tab.size(); ++j)
            if(tab.get(j) < tab.get(indMin))
                indMin = j;

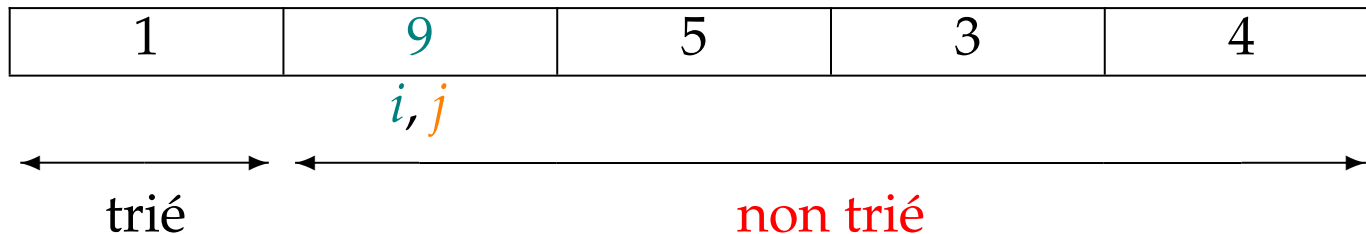
        //placement du minimum à sa place définitive
        int tmp = tab.get(i);
        tab.set(i, tab.get(indMin));
        tab.set(indMin, tmp);
    }
}
```

Tri par insertion (I)



A chaque étape i (de 1 à $n - 1$) :

- ① trouver la place relative j de l'élément i parmi les éléments triés
- ② déplacer l'élément i à la place j

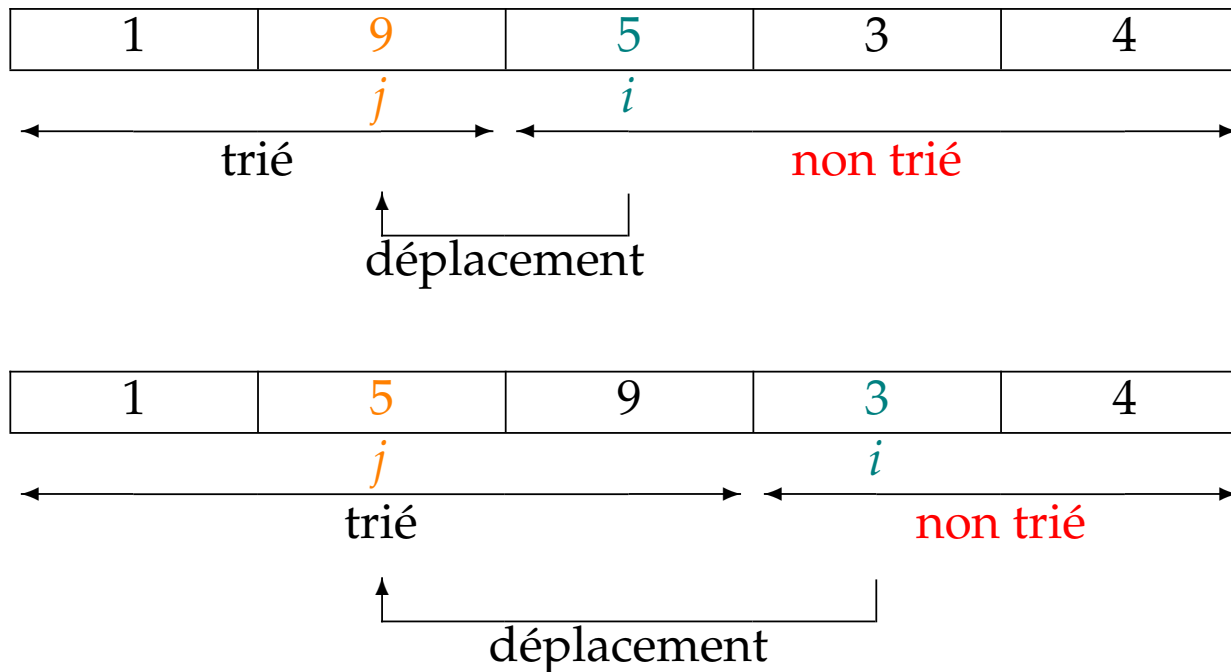


L'élément i est déjà à sa place

Tri par insertion (2)

A chaque étape i (de 1 à $n - 1$) :

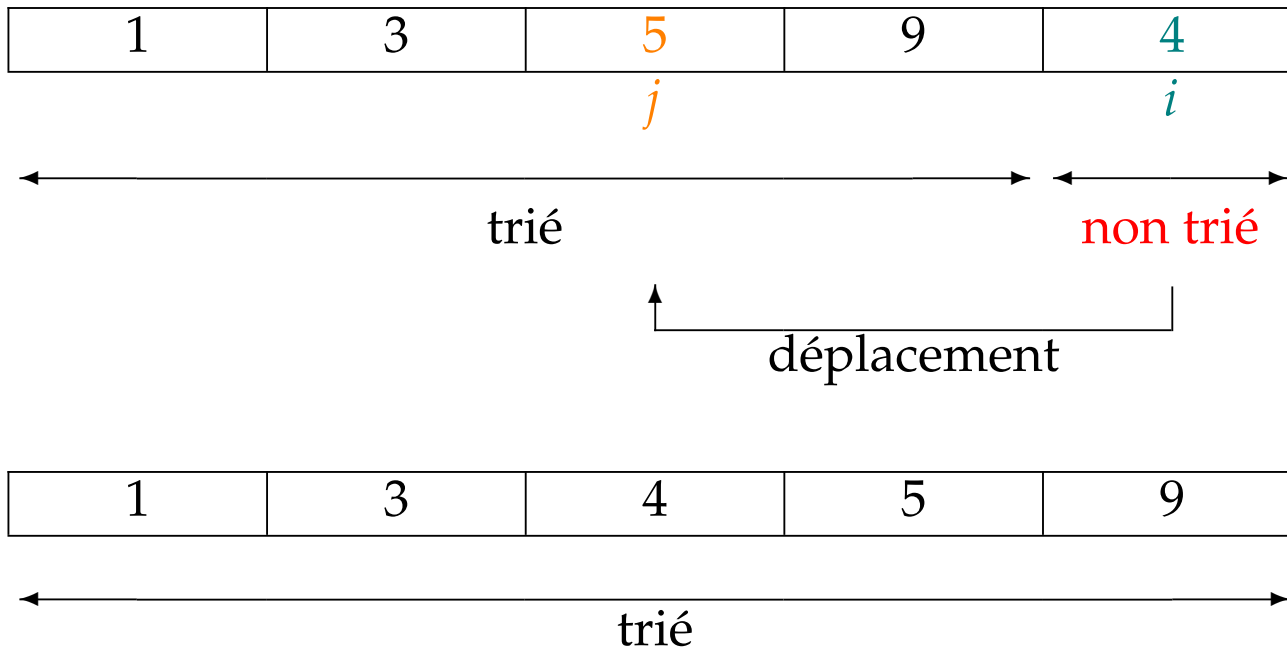
- ① trouver la place relative j de l'élément i parmi les éléments triés
- ② déplacer l'élément i à la place j



Tri par insertion (3)

A chaque étape i (de 1 à $n - 1$) :

- ① trouver la place relative j de l'élément i parmi les éléments triés
- ② déplacer l'élément i à la place j



Tri par insertion en Java pour un Array<Integer>

```

public static void triInsertion(Array<Integer> tab) {
    for(int i = 1; i < tab.size(); ++i) {

        int val = tab.get(i);
        int j = i;

        while(j > 0 && val < tab.get(j - 1)) {
            tab.set(j, tab.get(j-1)); //déplacement vers la droite
            --j;                      //tant que val < tab.get(j - 1)
        }

        //insertion
        tab.set(j, val);
    }
}
    
```

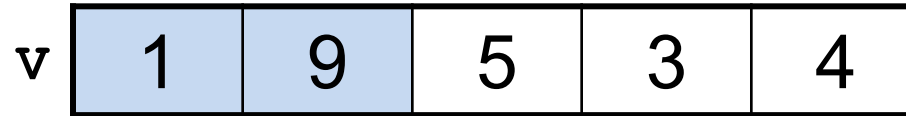
Tri bulle, tri par échange (I)

A chaque étape i (de $n-1$ à 1),

comparer deux à deux tous les éléments contigus du tableau entre 0 et i

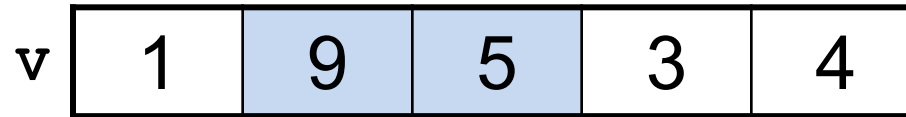
inverser ces éléments s'ils ne sont pas dans le bon ordre.

Tri bulle, tri par échange (2) première passe



j

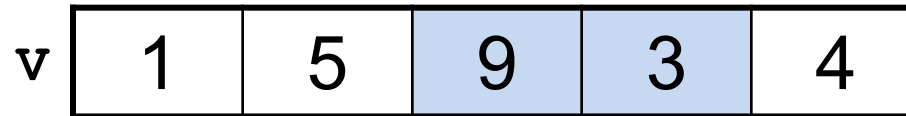
i



j

i

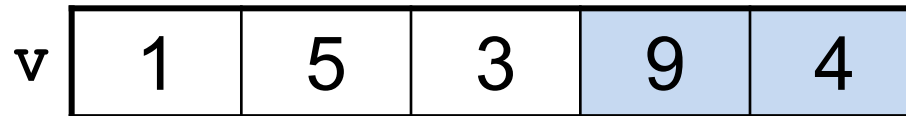
swap



j

i

swap

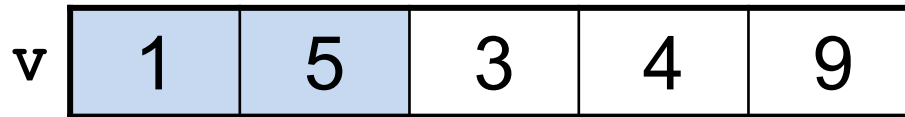


j

i

swap

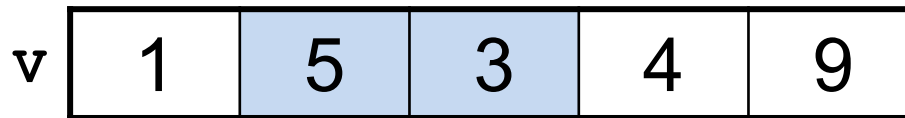
Tri bulle, tri par échange (2) seconde passe



j

i

On sait que le maximum est à la fin : il est à sa bonne place définitive.

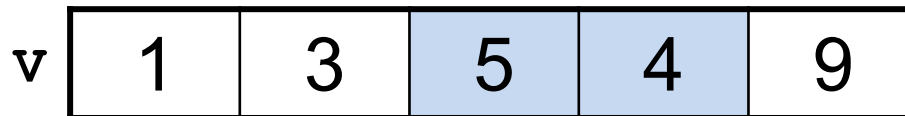


j

i



swap

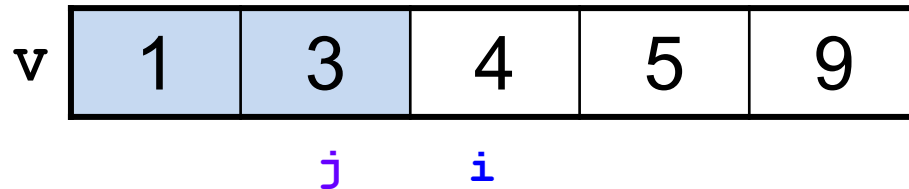


j i



swap

Tri bulle, tri par échange (3) troisième passe

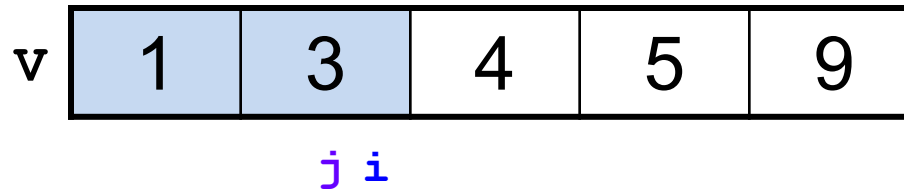


On sait que les deux plus grands sont à la fin. Ils sont à leur bonne place définitive.



Tri bulle, tri par échange (4) quatrième passe

On sait que les trois plus grands sont à la fin. Ils sont à leur bonne place définitive.



On sait que les quatres plus grands sont à la fin. Ils sont à leur bonne place définitive. Le premier est ainsi aussi rangé.

Tri bulle en Java pour un Array<Integer>

```

public static void triBulle(Array<Integer> tab) {
    for(int i = tab.size() - 1; i > 0; --i)
        for(int j = 0; j < i; ++j)
            if(tab.get(j) > tab.get(j+1)) {
                int tmp = tab.get(j);
                tab.set(j, tab.get(j + 1));
                tab.set(j + 1, tmp);
            }
}

```